

GraphPilot-Edge Revision Report

From Working Prototype to CASES-Grade Systems Paper

Comprehensive plan for novelty framing, calibrated simulation, rigorous evaluation, and next-scope execution

Purpose. This report incorporates the reviewer-style critique, restructures the paper contribution around stronger and more defensible claims, specifies the exact simulator and algorithmic upgrades required, defines the complete experiment and ablation matrix, and ends with an actionable work plan with tasks, acceptance criteria, and deliverables.

Document type:	Technical revision and execution plan
Target venues:	CASES / ESWeek, with portability toward PACT-style systems evaluation
System status anchor:	Current audited GraphPilot-Edge prototype with CPU+NPU deployment and real artifact pack
Output scope:	Revised paper blueprint + simulator plan + evaluation plan + task plan

March 12, 2026

Contents

1	Executive Summary	3
2	Current Prototype Anchor and Honest State	3
2.1	What the current system already proves	3
2.2	What the current system does <i>not</i> yet prove	4
3	Reviewer Critique Synthesized into a Concrete Revision Strategy	4
3.1	High-level revision outcomes required	4
4	Reframed Paper Thesis, Contributions, and Differentiation	4
4.1	The one-sentence thesis	4
4.2	Why this is unique	5
4.3	Comparison matrix to place in the paper	5
5	Revised Problem Formulation and Objective	5
5.1	Eight-term objective	6
5.2	How to make the objective defensible	6
5.3	Hard constraints	6
6	A Better Simulator: Public Anchors + Calibrated Surrogates	6
6.1	Hardware simulator structure	6
6.2	What should come from actual hardware data	7
6.3	Execution-time equation	7
6.4	Fallback-aware execution	8
6.5	Transfer, contention, batching, and thermal	8
6.6	Why this simulator is publishable	9
7	Model-Graph Simulator: Families, Knobs, and Stage Decompositions	9
7.1	LLMs	9
7.2	VLMs	9
7.3	STT and TTS	9
7.4	Embeddings, retrieval, CNN, ViT, and glue	10
8	Planning and Scheduling Algorithms	10
8.1	Why the problem is hard	10
8.2	Stage frontier extraction	10
8.3	Stage-level exhaustive search	10
8.4	Macro-region beam search	11
8.5	Continuous discrete-event simulation	11
8.6	Online dispatch rule	12
8.7	Memory and KV control	12

9	How to Make the Simulator “Very Good” in Practice	12
9.1	Step 1: measure stage families, not just end-to-end pipelines	12
9.2	Step 2: measure transfer and launch overhead directly	13
9.3	Step 3: fit thermal and contention parameters with sustained traces	13
9.4	Step 4: validate by family and by workflow	13
9.5	Step 5: report error honestly	13
10	Heavy Experiment Matrix Required for a Better Paper	13
10.1	Result block 0: current prototype anchor	13
10.2	Result block 1: simulator accuracy	13
10.3	Result block 2: fallback penalty quantification	14
10.4	Result block 3: baseline scheduling comparison	14
10.5	Result block 4: memory and KV pressure	14
10.6	Result block 5: thermal and plan-bank behavior	14
10.7	Result block 6: objective sensitivity	14
10.8	Result block 7: workload breadth and generality	14
11	Ablation Plan	15
12	How to Restructure the Paper	15
12.1	Recommended section order	15
12.2	Architecture figures that are essential	16
13	Concrete Related-Work Strategy	16
13.1	Communities to cover	16
13.2	One synthesis paragraph for the final paper	17
14	What Still Needs To Be Built, Exactly	17
14.1	Highest-value missing implementation pieces	17
14.2	What can remain out of scope without killing the paper	17
15	Detailed Next Work Plan and Task Breakdown	18
16	Risks and Mitigations	20
16.1	Risk 1: backend feasibility remains narrow	20
16.2	Risk 2: simulator error remains too high on some families	20
16.3	Risk 3: the project becomes too broad	20
17	Final Recommended Paper Story	20
	References	21

1. Executive Summary

GraphPilot-Edge already has something many systems papers do not: a real prototype, a real runtime, and real measured end-to-end workflows. The present artifact is not merely a simulator or a design sketch. It already executes three assistant-style workflows, preserves a support-safe FastVLM-on-NPU path, exposes live responder-to-TTS streaming, and contains offline planning and simulation infrastructure. However, the current paper material still reads like a design-and-status report rather than a submission-ready systems paper. The central gap is not that the system is fake; the gap is that the current draft demonstrates *what the system is* more clearly than *what the system proves*. This report closes that gap by redefining the paper around the strongest measurable contribution: a profiler-driven runtime and calibrated simulator for *continuous multimodal assistant DAGs on heterogeneous mobile SoCs*, with explicit support-safe fallback costing, LLM/VLM-specific memory and KV modeling, streaming-aware scheduling, and sim-to-real validation.

The key shift recommended here is to turn the simulator from background infrastructure into a visible contribution, to broaden the workload suite far beyond the current three workflows, and to add the small but crucial result blocks that would let CASES reviewers say “this is a real systems paper” rather than “this is promising future work.” In concrete terms, the paper must add five things: a validated multi-resource simulator, a proper related-work comparison table, a strong baseline suite, a fallback-penalty experiment, and a small but honest ablation matrix. The report below provides the exact formulas, algorithms, experiments, artifacts, and task breakdown necessary to complete that next scope.

2. Current Prototype Anchor and Honest State

The current prototype is already a meaningful and publishable foundation. It contains an offline planning stack, a discrete-event simulator, a planner, a runtime scheduler scaffold, memory and KV models, experiment scripts, and Android-side runtime integration. Most importantly, it also contains a real support-safe deployment path: FastVLM runs on LiteRT NPU, while ASR, planner, responder, retrieval, and TTS currently run on CPU in the best deployed plan. GPU retrieval exists as a support-safe alternative but loses to CPU retrieval in the current workflow-C measurements. This is not a weakness; it is exactly the kind of grounded result that makes the next paper credible.

2.1 What the current system already proves

- The project is not only offline simulation; there is a working end-to-end runtime.
- The system already supports compound DAG execution with one real parallel branch (VLM || retrieval).
- The system already contains a real streaming edge (responder $\rightarrow$ TTS).
- The current feasible backend set is narrow but honest. Support-safe constraints are being enforced rather than assumed away.
- The current artifact is therefore a solid “honest starting point” rather than a fictional systems claim.

2.2 What the current system does *not* yet prove

The prototype does not yet prove the strongest intended paper claim. It is not yet a fully online event-driven heterogeneous scheduler over arbitrary continuous mixed-criticality workloads. It does not yet contain the full calibrated multi-instance hardware simulator described in the idealized design. It does not yet contain the broad workload universe required to justify a general assistant-runtime claim. It does not yet compare against strong baseline classes from adjacent literature. It does not yet present sim-to-real accuracy metrics, fallback-penalty quantification, or a full ablation story. Those are the missing ingredients this report is designed to close.

3. Reviewer Critique Synthesized into a Concrete Revision Strategy

The critique is correct on the major points. The novelty is currently undersold, the experimental story is too light, the objective needs justification, the simulator needs its own validation section, and the current draft lacks related work, strong baseline comparison, and architecture figures. Rather than treating these as separate complaints, it is useful to compress them into one unified diagnosis:

Diagnosis. The current write-up describes a promising system but does not yet isolate the empirical phenomenon that the system uniquely explains, namely that on mobile SoCs naive heterogeneity breaks because unsupported-operation fallback, shared-memory contention, KV growth, and streaming dependencies interact in ways that classical mobile schedulers and LLM runtimes do not model jointly.

Once that diagnosis is accepted, the revision path becomes much clearer. The paper should lead with a measured motivating result about fallback-aware costing. It should then present the simulator and runtime as a joint response to that failure mode. The rest of the experiments should show that the same modeling framework also improves decisions under continuous arrivals, memory pressure, and thermal drift.

3.1 High-level revision outcomes required

1. Reframe the novelty around *support-safe fallback-aware scheduling of continuous assistant DAGs*.
2. Elevate the simulator to a standalone validated contribution.
3. Add a compact but strong baseline suite and paper-inspired baseline classes.
4. Add result blocks that directly support the central claim, rather than only describing future evaluation plans.
5. Tighten the claim to exactly what the completed system can defend.

4. Reframed Paper Thesis, Contributions, and Differentiation

4.1 The one-sentence thesis

Proposed paper thesis. *GraphPilot-Edge is a profiler-driven runtime and calibrated simulator for continuous multimodal assistant DAGs on heterogeneous mobile SoCs that jointly optimize support-safe placement, stage-specific speed-quality knobs, streaming overlap, and memory/KV policies under shared-memory, fallback, and thermal constraints.*

This thesis is stronger than the original broad formulation because it is specific where it needs to be specific. It does not claim universal heterogeneous scheduling for all stages on all backends. It claims a support-safe, calibrated, workload-aware runtime and simulator for a class of edge-AI systems that are both important and currently under-served by existing work.

4.2 Why this is unique

The cleanest differentiation is not “we also use CPU, GPU, and NPU.” Many systems do that. The differentiator is the *combination* of four properties:

1. **Compound assistant DAGs, not single-model inference.** Existing mobile runtimes and most LLM engines optimize one model at a time. GraphPilot-Edge reasons over STT, planner, VLM, retrieval, responder, and TTS together.
2. **Support-safe fallback-aware costing.** Existing heterogeneous schedulers usually assume that if a model is assigned to an accelerator, it runs there natively. On mobile SoCs this assumption is routinely false.
3. **Joint memory/KV/thermal reasoning.** Existing multi-DNN mobile schedulers do not capture LLM/VLM-specific KV cache growth and decode behavior, while LLM-serving systems typically assume homogeneous GPU servers.
4. **Continuous streaming and mixed-criticality execution.** The target is not one isolated request. The target is overlapping assistant traffic with foreground and background work, partial streaming edges, and response-time objectives like TTFS.

4.3 Comparison matrix to place in the paper

Property	HEFT/CPOP	Band/ADMS	vLLM/Orca	Puzzle/Twill	Agent.xpu
Heterogeneous mobile SoC	✓	✓	—	✓	✓
Compound assistant DAG	—	partial	—	partial	partial
Support-safe fallback costing	—	—	—	—	—
LLM KV-aware control	—	—	✓	—	✓
Streaming inter-stage edges	—	—	partial	partial	partial
Thermal / contention-aware runtime	—	partial	—	partial	partial
Knob co-optimization	—	—	—	—	partial
Sim-to-real calibration on mobile	—	—	—	partial	partial

Table 1: Proposed differentiation table for the paper. The goal is not to claim superiority on every dimension individually, but to show that GraphPilot-Edge is the first system in the overlap of these dimensions.

5. Revised Problem Formulation and Objective

A user request stream consists of a set of overlapping processes $r \in \mathcal{R}$, each associated with a directed acyclic graph $G_r = (V_r, E_r)$. Nodes are model stages or macro-regions. Edges carry outputs and can be of type FULL, CHUNK, or TOKEN. A plan assigns to each node a variant v , a hardware instance h , and a knob vector k .

5.1 Eight-term objective

The full scoring function should be:

$$J(\Pi) = \alpha \cdot P95(T_{e2e}) + \beta \cdot P95(T_{TFS}) + \gamma \cdot \bar{E} + \delta \cdot M_{peak} + \eta \cdot B_{copy} + \zeta \cdot Q_{loss} + \xi \cdot P95(T_{queue}) + \psi \cdot R_{miss}. \quad (1)$$

Here T_{e2e} is end-to-end latency, T_{TFS} is time to first speech, $\bar{E}$ is average energy or energy proxy, M_{peak} is peak memory, B_{copy} is copied bytes or copy time, Q_{loss} is quality loss against a reference configuration, T_{queue} is time spent waiting in scheduler queues, and R_{miss} is deadline miss rate. The current prototype already computes queue delay and miss-related statistics in stream simulation, but the current scorer does not yet fully integrate them into plan ranking. This must be fixed to align the math, simulator, and runtime claim.

5.2 How to make the objective defensible

The reviewer comment about weight fragility is correct. The paper should not present eight free weights with no explanation. There are two defensible options:

Option A: Frozen weighted objective. Pick weights once on a held-out calibration suite, then freeze them for all remaining results. The paper should report this explicitly and include a sensitivity plot where each weight is perturbed by $\pm 30\%$ while observing whether the top-ranked plan changes.

Option B: Lexicographic objective. Use TTFS as the primary key, then memory feasibility, then energy and queueing. For a mobile assistant this is intuitive and easier to defend. The weighted formulation can still be kept internally for tie-breaking.

5.3 Hard constraints

The plan is admissible only if:

$$M_{peak} \leq M_{budget}, \quad Q_{loss} \leq \epsilon_q, \quad R_{miss} \leq \epsilon_d, \quad (2)$$

and every placement decision is support-safe.

6. A Better Simulator: Public Anchors + Calibrated Surrogates

The simulator must become a first-class contribution. The critique was right that the current backend-level model is too coarse for the final paper. The upgraded simulator should have two layers: a hardware-resource simulator and a model-graph simulator.

6.1 Hardware simulator structure

Let the resource set be

$$\mathcal{H} = \mathcal{C} \cup \mathcal{G} \cup \mathcal{N}, \quad (3)$$

where $|\mathcal{C}| = n_C$, $|\mathcal{G}| = n_G$, and $|\mathcal{N}| = n_N$ are the counts of CPU, GPU, and NPU resource instances. Each resource instance $h \in \mathcal{H}$ has attributes

$$a(h) = (\text{type}_h, \Omega_h, \mu_h, BW_h^{mem}, BW_{h,h'}, L_h, P_h^{busy}, P_h^{idle}, Q_h^{max}, \Theta_h). \quad (4)$$

Here Ω_h is the support mask over operation classes, μ_h is a service-rate vector, BW_h^{mem} is effective memory bandwidth, $BW_{h,h'}$ is inter-resource transfer bandwidth, L_h is launch overhead, Q_h^{max} is queue depth, and Θ_h stores batching and thermal parameters.

6.2 What should come from actual hardware data

The simulator should use actual measured data wherever possible and only fit surrogates where direct measurement or vendor disclosure is unavailable.

Public hardware anchors. Use vendor- and runtime-documented facts as hard anchors: CPU cluster topology, supported memory configuration, NPU precision support, LiteRT/QNN support paths, and any officially documented queueing/compilation behavior.

Measured on-device quantities. Measure the following directly on the QIDK or target device:

- CPU microbenchmarks for GEMM, convolution, gather-heavy, ANN-heavy, and memcpy kernels on each cluster.
- GPU microbenchmarks for GEMM/conv/attention-like kernels, kernel launch cost, and transfer cost.
- NPU stage-level and operator-class throughput using LiteRT/QNN and real converted graphs, including unsupported-op detection.
- Transfer links CPU $\leftrightarrow$ GPU, CPU $\leftrightarrow$ NPU, GPU $\leftrightarrow$ NPU via synthetic tensor transfers and device-to-device chains.
- Thermal drift traces under sustained load for CPU, GPU, and NPU-dominant workloads.
- Memory-residency and RSS traces using device-side instrumentation during continuous mixed workloads.

Fitted surrogate parameters. Fit only the quantities that cannot be directly read or measured cleanly, such as per-op effective service rates after framework overhead, contention-coupling coefficients, or thermal time constants.

6.3 Execution-time equation

For task or region u on resource h , with batch size B , thermal state θ , and contention state q :

$$T_{u,h}^{exec} = L_h + \max(T_{u,h}^{ops}(B), T_{u,h}^{mem}) \cdot \rho_h(\theta) \cdot \kappa_h(q). \quad (5)$$

The compute term is:

$$T_{u,h}^{ops}(B) = \frac{1}{\phi_h(B)} \sum_o \frac{w_{u,o}}{\mu_{h,o}}, \quad (6)$$

where $w_{u,o}$ is the op-volume for operation class o and $\phi_h(B)$ is batch efficiency. The memory term is:

$$T_{u,h}^{mem} = \frac{m_u}{BW_h^{eff}}, \quad (7)$$

where m_u is the bytes touched by the task. Taking the max gives a roofline-style simulator: whichever dimension is slower dominates.

6.4 Fallback-aware execution

This is the centerpiece formula of the whole paper. If a region u is assigned to resource h but contains unsupported operations, partition it into maximal contiguous supported/unsupported chunks:

$$\pi(u, h) = \{r_1, r_2, \dots, r_m\}. \quad (8)$$

Then the actual execution time is:

$$T_{u,h}^{fb} = \sum_{r \in \pi(u,h)} T_{r,a(r)}^{comp} + \sum_{\ell=1}^{m-1} T_{r_\ell \rightarrow r_{\ell+1}}^{xfer}. \quad (9)$$

This formula is important enough that the final paper should motivate it with a measured trace: the optimistic “all-NPU” latency versus the actual latency once a hidden CPU fallback region and the two transfers are accounted for.

6.5 Transfer, contention, batching, and thermal

Transfer cost for tensor size S from resource h to h' is

$$T^{xfer}(S, h, h') = \begin{cases} 0, & h = h' \text{ and memory-compatible} \\ T^{map}(S), & \text{shared-buffer path exists} \\ \tau_{h,h'}^0 + \frac{S}{BW_{h,h'}} + \tau_{h,h'}^{layout}, & \text{otherwise.} \end{cases} \quad (10)$$

Contention is modeled as

$$\kappa_h(q) = 1 + \sum_{h' \in D_m(h) \setminus \{h\}} \lambda_{h,h'} u_{h'}, \quad (11)$$

where $D_m(h)$ is the memory-sharing domain of h and $u_{h'}$ is utilization.

A simple saturating batching model is

$$\phi_h(B) = 1 + \beta_h(1 - e^{-B/\tau_h}). \quad (12)$$

Thermal evolution for domain d should be modeled as

$$\theta_d(t + \Delta) = \theta_{amb} + (\theta_d(t) - \theta_{amb})e^{-\Delta/\tau_d} + R_d P_d(t)(1 - e^{-\Delta/\tau_d}), \quad (13)$$

with slowdown

$$\rho_h(\theta_d) = 1 + \gamma_h \max(0, \theta_d - \theta_h^{th}). \quad (14)$$

6.6 Why this simulator is publishable

The simulator becomes a real contribution once the paper reports prediction error on real device traces. The minimum publishable validation should show mean absolute percentage error (MAPE) for TTFT, TTFS, and end-to-end latency across at least three shapes per model family and across the three current workflows. A simulator with $< 15\%$ MAPE would already be useful; under 10% would be a strong result.

7. Model-Graph Simulator: Families, Knobs, and Stage Decompositions

The model-graph simulator should represent stages not as opaque boxes but as stage families with structure.

7.1 LLMs

Every LLM must be split into tokenizer, prefill, decode, and post-processing. Prefill and decode are not the same computational regime. An approximate cost decomposition is

$$T_{resp} = T_{prefill} + T_{switch} + N_{out} \cdot t_{decode}, \quad (15)$$

where the backend-switch penalty is

$$T_{switch} = \begin{cases} 0, & b_p = b_d \\ \tau_0 + \frac{M_{KV}}{BW_{b_p, b_d}}, & b_p \neq b_d. \end{cases} \quad (16)$$

KV size is

$$M_{KV} = 2LH_{kv}D_{head}TB_{dtype}. \quad (17)$$

Relevant knobs are context cap, max output tokens, retrieval token budget, quantization level, speculative decoding on/off, and prefill/decode backend selection.

7.2 VLMs

VLMs should be split into image preprocessing, vision encoder, projector/fusion, multimodal prefill, and decode or answer head. Visual token count must be modeled explicitly because it is one of the most powerful quality-speed levers. Relevant knobs are image resolution, crop count, visual-token budget, visual-token pruning ratio, OCR on/off, and max multimodal context.

7.3 STT and TTS

STT should be represented as audio frontend, encoder, decoder, and optional punctuation/intent post-processing. Its main knobs are chunk size, beam size, endpointing aggressiveness, quantization, and streaming-partial frequency. TTS should be represented as text normalization, acoustic model, and vocoder, with knobs such as chunk size, sample rate, vocoder quality mode, and sentence-splitting aggressiveness.

7.4 Embeddings, retrieval, CNN, ViT, and glue

Embeddings and ANN retrieval must be separate stages. ANN search should expose corpus size, top- k , and probe/efSearch style knobs. CNN and ViT workloads should expose input resolution, token or patch count, and precision. Sequential glue tasks—tokenization, image resize, audio framing, prompt assembly, JSON parsing, OCR wrappers, chunk packing—must be represented explicitly, because they can dominate TTFS even when they look small on paper.

8. Planning and Scheduling Algorithms

8.1 Why the problem is hard

The full problem is NP-hard in general because it subsumes heterogeneous DAG scheduling with communication costs, which is already a known hard problem. In practice the solution must therefore be heuristic but structured.

8.2 Stage frontier extraction

For each stage or macro-region u , generate all feasible (v, h, k) triples and retain only the non-dominated set with respect to latency, memory, energy, quality, and fallback risk.

Algorithm 1 Frontier extraction for one stage

Require: Feasible set F_u of triples (v, h, k) with measured or simulated attributes

Ensure: Non-dominated frontier $\mathcal{F}_u$

- 1: Sort F_u by latency, then memory, then energy
 - 2: $\mathcal{F}_u \leftarrow \emptyset$
 - 3: **for all** $x \in F_u$ **do**
 - 4: **if** there is no $y \in \mathcal{F}_u$ that dominates x **then**
 - 5: Remove all $z \in \mathcal{F}_u$ dominated by x
 - 6: Add x to $\mathcal{F}_u$
 - 7: **end if**
 - 8: **end for**
 - 9: **return** $\mathcal{F}_u$
-

This costs $O(|F_u| \log |F_u|)$ for sorting plus dominance checks depending on implementation.

8.3 Stage-level exhaustive search

At whole-stage granularity, brute force is still tractable. This is the right place to generate CPU-only, GPU-only where feasible, NPU-only where feasible, and stage-greedy baselines.

Algorithm 2 Stage-level exhaustive plan search

Require: Stage frontiers $\{\mathcal{F}_u\}_{u \in V}$

Ensure: Top- K stage-level plans

- 1: $\mathcal{P} \leftarrow$ Cartesian product of all stage frontiers
 - 2: **for all** $\Pi \in \mathcal{P}$ **do**
 - 3: Simulate Π over the workload set
 - 4: Score Π with objective $J(\Pi)$
 - 5: **end for**
 - 6: Return the top- K lowest-score plans
-

8.4 Macro-region beam search

Only the heavy stages should be opened internally, notably FastVLM and responder prefill/decode. Let beam width be B and the number of opened regions be D . Complexity is roughly $O(B \cdot D \cdot C)$ where C is the number of choices per region.

Algorithm 3 Macro-region beam search

Require: Ordered opened regions $r_1, \dots, r_D$, beam width B

Ensure: Top- K macro-aware plans

```

1: Beam  $\mathcal{B} \leftarrow \{\text{empty plan}\}$ 
2: for  $i = 1$  to  $D$  do
3:    $\mathcal{B}' \leftarrow \emptyset$ 
4:   for all  $\psi \in \mathcal{B}$  do
5:     for all choice  $c \in \mathcal{F}_{r_i}$  do
6:        $\psi' \leftarrow \psi$  extended with  $c$ 
7:       compute lower bound  $LB(\psi')$ 
8:       if  $\psi'$  is not dominated then
9:         add  $\psi'$  to  $\mathcal{B}'$ 
10:      end if
11:    end for
12:  end for
13:   $\mathcal{B} \leftarrow$  best  $B$  states in  $\mathcal{B}'$  by  $LB$ 
14: end for
15: Return best complete plans from  $\mathcal{B}$ 

```

8.5 Continuous discrete-event simulation

Algorithm 4 Discrete-event simulation for continuous mixed workloads

Require: Request stream, hardware state, memory state, candidate plan Π

Ensure: Simulated TTFT, TTFS, end-to-end, queue delay, miss rate, memory, energy

```

1: Initialize event queue with request arrivals
2: while event queue not empty do
3:   Pop earliest event
4:   if arrival event then
5:     create root tasks whose inputs are ready
6:   else if task-completion event then
7:     free expired buffers, update temperatures, update dependent tasks
8:   end if
9:   for all newly-ready tasks  $u$  do
10:    for all feasible resources  $h$  do
11:      compute  $EST(u, h)$  and  $EFT(u, h)$ 
12:      compute dynamic priority  $P(u, h)$ 
13:    end for
14:    dispatch best  $(u, h)$  if memory admission succeeds
15:  end for
16: end while

```

Here

$$EST(u, h) = \max(t_{deps}(u), t_{free}(h), t_{mem}(u, h), t_{qslot}(h)), \quad EFT(u, h) = EST(u, h) + T_{u,h}^{exec}. \quad (18)$$

8.6 Online dispatch rule

The online runtime should use a dynamic priority function of the form

$$\begin{aligned} P(u, h) = & w_r \text{rank}_u(u) + w_f F(u) + w_a \text{age}(u) + w_s \text{slack}(u) \\ & + w_{aff} \text{Aff}(u, h) - w_c \text{Xfer}(u, h) - w_m \text{MemRisk}(u, h) \\ & - w_t \text{ThermRisk}(h) - w_l \text{LateRisk}(u, h). \end{aligned} \quad (19)$$

with HEFT-style critical-path rank

$$\text{rank}_u(i) = \bar{T}_i + \max_{j \in \text{Succ}(i)} (\bar{C}_{i \rightarrow j} + \text{rank}_u(j)). \quad (20)$$

Dispatch chooses the highest-priority pair, tie-broken by smallest earliest finish time.

8.7 Memory and KV control

KV admission should obey

$$\sum_s M_{KV}(s) + M_{workbuf} + M_{live} \leq M_{budget} - M_{margin}. \quad (21)$$

If violated, choose the degradation action

$$a^* = \arg \min_a \frac{\Delta Q(a) + \lambda_L \Delta L(a)}{\Delta M(a) + \epsilon}. \quad (22)$$

Decode should remain sticky to its current backend unless

$$T_{remain}(b_{cur}) - T_{remain}(b_{new}) > T_{KV-migrate} + H. \quad (23)$$

9. How to Make the Simulator “Very Good” in Practice

The user request explicitly asked for simulator quality using actual CPU, GPU, and NPU data. The right answer is not to pretend the vendor publishes every parameter. The right answer is a layered calibration workflow.

9.1 Step 1: measure stage families, not just end-to-end pipelines

Collect microbenchmarks and stage benchmarks for each family:

- LLM: prefill and decode separately, across prompt lengths and output lengths.
- VLM: encoder, projector, multimodal prefill, and answer head separately.
- STT: chunked streaming and full-utterance, with varying chunk sizes.
- TTS: first chunk, sustained chunk, acoustic model, and vocoder separately.
- Retrieval: embed, ANN, rerank, and chunk pack separately.
- CNN/ViT: conv-heavy and token-heavy variants across input sizes.
- Glue tasks: image resize, audio framing, tokenizer, prompt assembly, JSON parsing.

9.2 Step 2: measure transfer and launch overhead directly

A large source of simulator error on mobile SoCs is not the model math; it is the launch and transfer overhead that classical models ignore. Run synthetic transfer microbenchmarks at the tensor sizes observed in real workloads. Also measure the fixed dispatch overhead for CPU, GPU, and NPU launches. These are crucial for STT/TTS chunking studies.

9.3 Step 3: fit thermal and contention parameters with sustained traces

Run 20–30 minute sustained scenarios and fit τ_d , R_d , and contention couplings $\lambda_{h,h'}$ from measured slowdown traces. One good thermal trace is worth more than ten pages of theoretical thermal prose.

9.4 Step 4: validate by family and by workflow

Do not validate only at end-to-end level. Validate at two levels:

1. **Stage-family level:** predicted versus measured latency for at least three shapes per family.
2. **Workflow level:** predicted versus measured TTFT/TTFS/end-to-end for A/B/C and at least one bursty continuous trace.

9.5 Step 5: report error honestly

The simulator section should include a table of MAPE values and one figure showing predicted vs measured scatter. If the simulator is excellent for VLM and weaker for STT chunking or TTS first-chunk, say so directly and explain why. That honesty increases rather than decreases trust.

10. Heavy Experiment Matrix Required for a Better Paper

This section converts the critique into a concrete experimental campaign.

10.1 Result block 0: current prototype anchor

Keep the current measured numbers as a preliminary-results anchor. Present them confidently as the starting point, not apologetically. Add peak-memory measurements alongside latency, because for mobile systems memory is a first-class result.

10.2 Result block 1: simulator accuracy

Run simulator-vs-real validation on:

- LLM prefill at three prompt lengths,
- LLM decode at three output-length regimes,
- VLM encoder at three visual-token budgets,
- STT at three chunk sizes,
- TTS at three chunk sizes,
- full workflows A/B/C.

Report MAPE for TTFT, TTFS, end-to-end latency, and peak memory where possible.

10.3 Result block 2: fallback penalty quantification

This is the fastest path to a compelling result. For each workflow, compare:

1. optimistic NPU plan assuming no fallback,
2. actual fallback-aware plan cost,
3. measured execution if feasible.

A single bar chart here may do more for acceptance than many pages of prose.

10.4 Result block 3: baseline scheduling comparison

At minimum compare:

- CPU-only,
- GPU-only where feasible,
- NPU-only where feasible,
- current deployed plan,
- StageGreedy,
- StaticBestMap,
- full GraphPilot-Edge.

Metrics: TTFT, TTFS, end-to-end latency, peak memory, copied bytes, and energy proxy.

10.5 Result block 4: memory and KV pressure

Use bursty multi-request traces to compare:

- GraphPilot memory/KV admission and degradation,
- naive scheduler with no explicit memory/KV control.

Report memory traces, OOM or reject events, TTFS degradation, and throughput under load.

10.6 Result block 5: thermal and plan-bank behavior

Show a sustained run with at least one thermal event. Compare static best-at-time-zero planning to GraphPilot plan-bank switching. Report switching overhead and latency impact.

10.7 Result block 6: objective sensitivity

Perturb the objective weights by $\pm 30\%$. Show whether the top-ranked plan changes and whether the best plan family is stable. If it is unstable, move to a lexicographic objective and say so.

10.8 Result block 7: workload breadth and generality

Include at least one representative result from each workload family class:

- LLM-only,
- VLM-heavy,

- STT/TTS-heavy,
- RAG-heavy,
- CNN/ViT-heavy,
- mixed foreground/background,
- unsupported-op stress,
- thermal stress.

This can be concise but it demonstrates breadth.

11. Ablation Plan

The current draft should not promise twenty future ablations without results. Instead it should run a focused and interpretable ablation suite.

Ablation	What is removed	What should change if the design matters
NoFallbackAware	fallback partition and copy penalties	Planner chooses fake accelerator-heavy plans; sim-to-real error rises; latency underestimation increases
NoPrefillDecodeSplit	separate text-stage modeling	Wrong backend selection for short vs long outputs; decode migration logic collapses
NoPipeline	CHUNK/TOKEN edge exploitation	TTFS degrades sharply while total end-to-end may degrade modestly
NoBranchOverlap	VLM retrieval overlap	workflow-C end-to-end grows and accelerator utilization drops
NoMemoryReuse	interval-based buffer allocator	peak memory increases and admissible concurrency drops
NoKVAdmission	explicit KV budget logic	burst scenarios show OOM or uncontrolled TTFS spikes
NoThermalBank	bank switching and hysteresis	sustained load degrades more than static profiling predicts
NoKnobTuning	visual-token / max-token / chunk knobs fixed	method loses robustness across workload regimes
NoFirstOutputBias	TTFS-aware dispatch terms removed	queueing may improve throughput while hurting perceived responsiveness

Table 2: Recommended ablation matrix. The paper does not need every one of these on day one, but it must include enough of them to demonstrate causal value of the system components.

12. How to Restructure the Paper

The draft should be restructured around evidence instead of around aspiration.

12.1 Recommended section order

1. **Introduction** with one measured motivating example about fallback error.
2. **Related Work** with the comparison matrix.

3. **Prototype Anchor and Preliminary Results** showing the current real system and its current best deployed plan.
4. **Problem Formulation and Design Goals** with notation table moved into main text.
5. **Hardware and Model-Graph Simulator** as a visible contribution.
6. **Planning and Runtime Algorithms** with complexity discussion.
7. **Evaluation** with the result blocks listed above.
8. **Ablations and Limitations**.
9. **Conclusion**.

12.2 Architecture figures that are essential

Two architecture figures should be added immediately:

Figure 1: Assistant DAG on a mobile SoC. A diagram showing $\text{ASR} \rightarrow \text{Planner} \rightarrow (\text{VLM} \parallel \text{Retrieval}) \rightarrow \text{Responder} \rightarrow \text{TTS}$, with CPU/GPU/NPU blocks and current support-safe placements highlighted.

Figure 2: Offline/online split. A diagram showing $\text{profiler} \rightarrow \text{simulator} \rightarrow \text{planner} \rightarrow \text{plan bank offline}$, and $\text{runtime scheduler} + \text{memory/KV controller} + \text{streaming edges online}$.

These do not need to be artistically heroic. They need to be clear and immediately legible.

13. Concrete Related-Work Strategy

The review comment on related work is exactly right: the current paper is missing a full section. The paper should not summarize papers one by one in isolation. Instead it should walk the reader through the six adjacent communities and end each subsection by explicitly saying what GraphPilot-Edge adds beyond them.

13.1 Communities to cover

- **Classical heterogeneous scheduling:** HEFT, CPOP, list scheduling. These assume static execution costs and operation-complete backends.
- **Mobile inference runtimes:** LiteRT, CoreML, MLC-LLM. These optimize single-model execution but not compound DAGs.
- **LLM serving systems:** Orca, vLLM, Splitwise-style prefill/decode separation. These handle KV and batching but assume server-class homogeneous resources.
- **Multi-DNN mobile scheduling:** Band, ADMS, Puzzle. These are your closest systems neighbors and need the most careful differentiation.
- **Compound AI systems:** Twill and related compound-AI schedulers. These broaden beyond DNN-only workloads but still lack your fallback/KV/assistant emphasis.
- **Agentic mobile systems:** Agent.xpu and HeRo. These are particularly relevant because they target agentic flows and mobile SoCs, but they do not simultaneously claim support-safe fallback costing, multimodal assistant DAGs with STT/TTS, and the same memory/KV + simulator story.

13.2 One synthesis paragraph for the final paper

A paragraph like the following should end the related-work section:

Classical heterogeneous schedulers handle static DAGs without fallback awareness. Mobile inference runtimes handle single models without pipeline orchestration. LLM serving systems handle KV memory without heterogeneous mobile placement or compound multimodal pipelines. Multi-DNN mobile schedulers handle placement without generative-model structures, stream-aware edges, or support-safe fallback costing. No prior system addresses the combination of fallback-aware heterogeneity, streaming assistant DAGs, LLM/VLM-specific memory modeling, and sim-to-real calibrated scheduling that characterizes GraphPilot-Edge.

14. What Still Needs To Be Built, Exactly

This section translates the revision plan into engineering scope.

14.1 Highest-value missing implementation pieces

1. Full multi-resource hardware simulator with calibrated service-rate vectors and support masks.
2. Proper per-stage frontier extraction before planning.
3. Full integration of queue delay and miss rate into plan scoring.
4. Runtime chunked STT $\rightarrow$ planner streaming.
5. Runtime thermal plan-bank switching with hysteresis and measurable switching overhead.
6. Stronger online backend queues and continuous mixed-criticality scheduling.
7. Real memory/KV controller integration in the deployed runtime, not only in offline models.
8. Baseline-class reproductions inside the same environment.
9. Simulator validation block with MAPE reporting.
10. Fallback-penalty motivating experiment and figure.

14.2 What can remain out of scope without killing the paper

The paper does *not* need to complete every dreamed feature to be strong. It can remain strong without all of the following as long as the claim is scoped correctly:

- universal support-safe GPU/NPU text execution for all text stages,
- arbitrarily fine operator-level partitioning,
- online global reoptimization at every event,
- distributed multi-device scheduling,
- RL-based policy learning.

Keeping those out of scope is often a strength.

15. Detailed Next Work Plan and Task Breakdown

The next scope should be executed in a strict order. Each task below includes an acceptance criterion so that the project does not drift.

ID	Task	What must be done	Acceptance criterion
T1	Revalidate current audited prototype	Re-run workflows A/B/C, confirm current best deployed plan, confirm live responder→TTS streaming, and lock a new branch for expansion work.	Current metrics and artifacts reproduce within tolerance; no regression.
T2	Add preliminary-results section inputs	Export current TTFT/TTFS/end-to-end and peak-memory measurements to a clean table that the paper can cite.	One canonical table for current prototype results exists and is checked.
T3	Implement multi-resource simulator core	Add explicit CPU/GPU/NPU instance objects, queue depth, transfer graph, launch overhead, support masks, batching, and thermal state.	Unit tests cover queueing, transfer, and thermal state updates.
T4	Implement fallback partitions	Given unsupported-op metadata, partition stages into supported/unsupported regions and apply fallback cost formula.	Optimistic-vs-fallback plan latency differs on at least one real workload and is logged.
T5	Build stage-family benchmark suite	Add model-family microbenchmarks for LLM prefill/decode, VLM encoder/projector, STT, TTS, retrieval, CNN, ViT, and glue tasks.	Profiler registry contains measurements for all required stage families.
T6	Fit simulator surrogates	Fit service-rate, contention, transfer-overhead, and thermal parameters using QIDK traces.	Calibration script outputs parameter files and MAPE report.
T7	Add frontier extraction	Implement non-dominated frontier generation per stage/variant/backend/knob.	Frontier size shrinks candidate count while preserving best measured plan.
T8	Integrate full 8-term objective	Add queue delay and miss rate into plan scoring and report sensitivity.	Objective scorer matches simulator metrics and weight sweep report exists.

ID	Task	What must be done	Acceptance criterion
T9	Add chunked STT→planner streaming	Start planner on partial STT output where feasible and measurable.	TTFS improves on at least one voice workload or the limitation is documented.
T10	Add runtime thermal plan-bank switching	Implement EWMA slowdown tracking and hysteresis-based switching across cool/warm/hot/lowmem banks.	Synthetic or real thermal trace shows at least one correct plan switch.
T11	Strengthen memory/KV controller	Connect admission, degrade/reject, buffer reuse, and decode stickiness into the live runtime.	Burst trace shows bounded memory without OOM and with logged admit/degrade decisions.
T12	Implement baseline suite	CPU-only, GPU-only where feasible, NPU-only where feasible, current deployed plan, StageGreedy, StaticBestMap, NoPipeline, NoFallbackAware, NoMemoryKV, NoKnobTuning, NoThermalAdaptation.	All baselines run inside the same harness and produce comparable tables.
T13	Implement paper-inspired baseline classes	Band-like, ADMS-like, Puzzle-like, Twill-like, HeteroInfer-like, Agent.xpu-like, HeRo-like method classes.	At least simplified faithful reproductions exist and are documented.
T14	Run fallback-penalty study	Quantify optimistic-NPU vs fallback-aware vs measured latency on representative workloads.	One figure and one table exist and support the paper’s motivating claim.
T15	Run simulator-validation study	Produce predicted-vs-measured results for model families and workflows.	MAPE table and scatter plot exist.
T16	Run continuous mixed-workload study	Evaluate Poisson, burst, and mixed foreground/background traces.	Queue delay, miss rate, TTFS, and throughput curves exist.
T17	Run heavy characterization studies	Backend affinity, shape sensitivity, launch overhead, transfer matrix, thermal drift, memory/KV growth, knob frontiers.	Characterization subsection is fully populated with plots/tables.

ID	Task	What must be done	Acceptance criterion
T18	Run ablation suite	Remove one component at a time from the full method.	At least 5 strong ablations are complete and interpretable.
T19	Add paper figures and related work table	Create architecture figures and place comparison matrix in the main draft.	Draft contains both figures and the comparison table.
T20	Rewrite abstract and introduction	Lead with a measured motivating example and scoped contribution claim.	New abstract and intro align with measured evidence and reviewer comments.
T21	Finalize artifact pack	Build new audited pack containing all simulator parameters, results, plots, reports, and draft.	Final pack is complete and auditable.

16. Risks and Mitigations

16.1 Risk 1: backend feasibility remains narrow

This is manageable. The paper does not require universal backend support if the claim is scoped around support-safe feasible execution plus calibrated simulation of broader possibilities. The mitigation is to be explicit about which results are online-realized and which are simulator-supported projections.

16.2 Risk 2: simulator error remains too high on some families

Also manageable. The mitigation is to report the error honestly, explain where the model is less predictive, and show that plan ranking remains directionally useful even when point prediction is imperfect.

16.3 Risk 3: the project becomes too broad

The mitigation is strict scope control: prioritize fallback penalty, simulator validation, current-plan baselines, and a handful of strong ablations before chasing exotic workload extensions.

17. Final Recommended Paper Story

The final paper should not read like “we built a huge future-facing runtime for everything.” It should read like this:

On heterogeneous mobile SoCs, naive accelerator-centric scheduling breaks because unsupported-operation fallback, transfer overhead, shared-memory contention, KV-cache

growth, and streaming dependencies interact in ways classical schedulers and single-model runtimes do not model. GraphPilot-Edge addresses this with a support-safe profiler-driven planner, a calibrated continuous-workload simulator, and a runtime that prioritizes streaming responsiveness and memory feasibility for multimodal assistant DAGs.

If the paper can prove that thesis with even a modest but honest set of measurements—especially simulator accuracy, fallback-penalty quantification, and continuous-load improvements over the right baselines—it becomes a strong and distinctive CASES paper.

References

References

- [1] Qualcomm, “Snapdragon 8 Elite Mobile Platform Product Brief,” official product brief, 2024–2025 revisions.
- [2] Google AI Edge, “Qualcomm NPU (AI Engine Direct) with LiteRT,” official LiteRT documentation, 2026.
- [3] J. S. Jeong et al., “Band: Coordinated Multi-DNN Inference on Heterogeneous Mobile Processors,” *MobiSys*, 2022.
- [4] Y. Gao et al., “Optimizing Multi-DNN Inference on Mobile Devices through Heterogeneous Processor Co-Execution,” arXiv:2503.21109, 2025.
- [5] L. Chen et al., “HeteroLLM / HeteroInfer: Accelerating Large Language Model Inference on Mobile SoCs with Heterogeneous AI Accelerators,” arXiv:2501.14794, 2025.
- [6] Z. Taufique et al., “Twill: Scheduling Compound AI Systems on Heterogeneous Mobile Edge Platforms,” arXiv:2507.00491, 2025.
- [7] X. Wei et al., “Agent.xpu: Efficient Scheduling of Agentic LLM Workloads on Heterogeneous SoC,” arXiv:2506.24045, 2025.
- [8] D. Kang et al., “Puzzle: Scheduling Multiple Deep Learning Models on Mobile Device with Heterogeneous Processors,” arXiv:2508.17764, 2025.
- [9] M. Li et al., “HeRo: Adaptive Orchestration of Agentic RAG on Heterogeneous Mobile SoC,” arXiv:2603.01661, 2026.
- [10] G. Yu et al., “Orca: A Distributed Serving System for Transformer-Based Generative Models,” OSDI, 2022.
- [11] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP, 2023.